



**ECOLE MAROCAINE DES  
SCIENCES DE L'INGENIEUR**

Membre de  
**HONORIS UNITED UNIVERSITIES**

*Ecole Reconnue par l'Etat*

# Rapport de Projet

## Deep Learning

EMSI - Année Universitaire 2025-2026

Ce rapport présente les travaux réalisés dans le cadre du projet de Deep Learning. Il couvre trois parties principales :

Partie I : MLP et PyTorch

Partie II : CNN et Vision par Ordinateur

Partie III : RNN, LSTM, GRU et Seq2Seq

**Elabore par : Yassir Ouahla**

**Technologies utilisées : Python, PyTorch, scikit-learn, NumPy, Matplotlib**

# Table des Matieres

## Partie I - MLP et PyTorch

- 1.1 Objectif
- 1.2 Dataset : Breast Cancer Wisconsin
- 1.3 Preprocessing des donnees
- 1.4 Architecture du MLP
- 1.5 Entrainement et evaluation
- 1.6 Comparaison des techniques d'initialisation
- 1.7 Analyse critique et conclusion

## Partie II - CNN et Vision par Ordinateur

- 2.1 Objectif
- 2.2 Dataset : Fashion-MNIST
- 2.3 Operations fondamentales (Convolution, Pooling)
- 2.4 Architecture du CNN
- 2.5 Entrainement et evaluation
- 2.6 Analyse critique et conclusion

## Partie III - RNN, LSTM, GRU et Seq2Seq

- 3.1 Objectif
- 3.2 Dataset : 20 Newsgroups
- 3.3 Pretraitement du texte et vocabulaire
- 3.4 Architectures : RNN, LSTM, GRU
- 3.5 Entrainement et comparaison
- 3.6 Mini modele Seq2Seq
- 3.7 Analyse critique et conclusion

# Partie I - MLP et PyTorch

## 1.1 Objectif

Dans cette premiere partie, l'objectif est de :

- Preparer un dataset tabulaire reel pour la classification binaire
- Construire un Perceptron Multicouche (MLP) avec PyTorch
- Entraîner et evaluer le modele avec des metriques pertinentes
- Comparer differentes techniques d'initialisation des poids
- Sauvegarder et recharger le modele entraine

## 1.2 Dataset : Breast Cancer Wisconsin

Le dataset utilise est le Breast Cancer Wisconsin, disponible via scikit-learn. Il contient 569 echantillons avec 30 caracteristiques numeriques chacun. La tache consiste a classer les tumeurs comme malignes (0) ou benignes (1).

| Propriete             | Valeur           |
|-----------------------|------------------|
| Nombre d'echantillons | 569              |
| Nombre de features    | 30               |
| Classes               | 2 (malin, benin) |
| Split Train/Test      | 80% / 20%        |
| Stratification        | Oui              |

## 1.3 Preprocessing des donnees

Les etapes de preprocessing appliquees sont :

- Separation train/test (80/20) avec stratification
- Normalisation avec StandardScaler (moyenne=0, ecart-type=1)
- Conversion en tenseurs PyTorch (float32 pour X, long pour y)
- Envoi vers le device (CPU ou GPU)

*La normalisation est essentielle pour la stabilite de l'apprentissage car les features ont des echelles tres differentes (ex: mean radius vs mean fractal dimension).*

## 1.4 Architecture du MLP

Le modele MLP est defini comme une classe heritant de nn.Module avec :

| Couche       | Entree | Sortie | Activation |
|--------------|--------|--------|------------|
| fc1 (Linear) | 30     | 64     | ReLU       |
| fc2 (Linear) | 64     | 32     | ReLU       |

|              |    |   |   |
|--------------|----|---|---|
| fc3 (Linear) | 32 | 2 | - |
|--------------|----|---|---|

Fonction de cout : CrossEntropyLoss

Optimiseur : Adam (lr=0.001)

Nombre d'epochs : 100

## 1.5 Entrainement et Evaluation

Le modele est entraine pendant 100 epochs. Les resultats montrent une convergence rapide de la loss et une accuracy d'entrainement atteignant 99.12% a l'epoch 100.

### Resultats sur le jeu de test

| Metrique  | Valeur |
|-----------|--------|
| Accuracy  | 0.9561 |
| Precision | 0.9718 |
| Recall    | 0.9583 |
| F1-Score  | 0.9650 |

La matrice de confusion montre que le modele classe correctement la grande majorite des echantillons, avec tres peu de faux positifs et faux negatifs.

## 1.6 Comparaison des Techniques d'Initialisation

Quatre techniques d'initialisation ont ete comparees sur le meme modele MLP :

| Initialisation | Test Accuracy | Test Precision | Test Recall | Test F1 |
|----------------|---------------|----------------|-------------|---------|
| Xavier         | 0.9561        | 0.9718         | 0.9583      | 0.9650  |
| He (Kaiming)   | 0.9386        | 0.9851         | 0.9167      | 0.9496  |
| Random Uniform | 0.9298        | 0.9324         | 0.9583      | 0.9452  |
| Normal         | 0.9298        | 0.9444         | 0.9444      | 0.9444  |

L'initialisation Xavier donne les meilleurs resultats globaux. L'initialisation He est optimisee pour ReLU et montre la meilleure precision. Les initialisations aleatoires (Random et Normal) sont les moins performantes.

## 1.7 Analyse Critique et Conclusion

Cette implementation demontre qu'un MLP constitue une solution efficace pour la classification de donnees tabulaires. Les points cles sont :

- L'importance du preprocessing (normalisation)
- Le role crucial de l'initialisation des poids
- L'impact des fonctions d'activation

- L'utilisation correcte de PyTorch et de l'API nn.Module

Le modele sauvegarde peut etre recharge et reutilise pour des predictions futures, demontrant la persistance des modeles PyTorch.

## Partie II - CNN et Vision par Ordinateur

### 2.1 Objectif

Cette deuxième partie vise à :

- Comprendre les opérations de convolution et de pooling
- Construire un CNN (Convolutional Neural Network) avec PyTorch
- Entraîner le modèle sur un dataset d'images réelles
- Évaluer et analyser les performances

### 2.2 Dataset : Fashion-MNIST

Le dataset Fashion-MNIST est utilisé, contenant 70 000 images en niveaux de gris de 28x28 pixels réparties en 10 catégories de vêtements et accessoires.

| Propriété         | Valeur      |
|-------------------|-------------|
| Train samples     | 60 000      |
| Test samples      | 10 000      |
| Taille des images | 28 x 28 x 1 |
| Nombre de classes | 10          |
| Batch size        | 64          |

Les 10 catégories sont : T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

### 2.3 Opérations Fondamentales

#### Convolution 2D (corr2d)

Une implémentation manuelle de la corrélation croisée 2D a été réalisée pour illustrer le fonctionnement des filtres convolutionnels. Cette opération applique un noyau (kernel) sur une matrice d'entrée pour extraire des features.

#### Max Pooling 2D

Le max pooling réduit la dimension spatiale en gardant la valeur maximale dans chaque fenêtre. Cela permet de réduire les paramètres et d'apporter une invariance aux petites translations.

### 2.4 Architecture du CNN

Le CNN utilise l'architecture suivante :

| Couche    | Configuration             | Sortie       |
|-----------|---------------------------|--------------|
| Conv2d #1 | 1->32 filtres, 3x3, pad=1 | 32 x 28 x 28 |

|                |                            |              |
|----------------|----------------------------|--------------|
| ReLU + MaxPool | 2x2                        | 32 x 14 x 14 |
| Conv2d #2      | 32->64 filtres, 3x3, pad=1 | 64 x 14 x 14 |
| ReLU + MaxPool | 2x2                        | 64 x 7 x 7   |
| Flatten        | -                          | 3136         |
| Linear #1      | 3136 -> 128                | 128          |
| ReLU           | -                          | 128          |
| Linear #2      | 128 -> 10                  | 10           |

## 2.5 Entrainement et Evaluation

Le modele est entraine pendant 5 epochs avec Adam (lr=0.001) et CrossEntropyLoss.

| Epoch | Loss   | Train Accuracy |
|-------|--------|----------------|
| 1     | 0.4713 | 82.93%         |
| 2     | 0.2969 | 89.24%         |
| 3     | 0.2500 | 90.80%         |
| 4     | 0.2237 | 91.80%         |
| 5     | 0.1957 | 92.77%         |

Test Accuracy finale : ~91.5%

Le modele montre une convergence rapide et atteint de bonnes performances en seulement 5 epochs. Le classification report detaille montre des variations de performance selon les categories, certaines (comme Trouser) etant plus faciles a classifier que d'autres (comme Shirt).

## 2.6 Analyse Critique et Conclusion

Points positifs :

- Le CNN extrait automatiquement des features spatiales pertinentes
- Performance elevee avec une architecture relativement simple
- Convergence rapide grace a Adam et au batch processing

Limites :

- Pas d'augmentation de donnees appliquee
- Architecture simple (2 couches conv) - des architectures plus profondes (ResNet, VGG) pourraient ameliorer les resultats
- Pas de regularisation (Dropout, BatchNorm)

Le modele CNN est sauvegarde et peut etre recharge pour une utilisation future.

## Partie III - RNN, LSTM, GRU et Seq2Seq

### 3.1 Objectif

Cette troisième partie vise à :

- Comprendre les modèles séquentiels (RNN, LSTM, GRU)
- Préparer des données textuelles réelles
- Implémenter et comparer les trois architectures
- Construire un mini système Seq2Seq

### 3.2 Dataset : 20 Newsgroups

Le dataset 20 Newsgroups de scikit-learn est utilisé. Il contient environ 18 846 messages provenant de 20 groupes de discussion Usenet. Pour cette tâche de classification, 4 catégories sont sélectionnées :

| Categorie             | Domaine                  |
|-----------------------|--------------------------|
| sci.space             | Science / Espace         |
| rec.sport.hockey      | Sport / Hockey           |
| comp.graphics         | Informatique / Graphisme |
| talk.politics.mideast | Politique / Moyen-Orient |

Le nettoyage automatique des métadonnées (headers, footers, quotes) est appliqué lors du chargement pour éviter les biais.

*Ce dataset réel contient ~2 300 échantillons d'entraînement et ~1 500 de test, ce qui constitue un défi plus réaliste qu'un petit dataset synthétique.*

### 3.3 Prétraitement du Texte et Vocabulaire

Les étapes de prétraitement sont :

1. Conversion en minuscules
2. Suppression des caractères spéciaux et chiffres (regex)
3. Tokenisation par mots
4. Filtrage des mots trop courts (< 2 caractères)
5. Construction du vocabulaire (mots freq >= 2)
6. Encodage en séquences d'indices avec padding/troncature

Le vocabulaire contient plusieurs milliers de mots. L'index 0 est réservé au padding (<PAD>) et l'index 1 aux mots inconnus (<UNK>). La longueur maximale des séquences est fixée à 200 tokens.

### 3.4 Architectures : RNN, LSTM, GRU



Les trois architectures partagent une structure commune :

| Composant         | Configuration        |
|-------------------|----------------------|
| Embedding         | vocab_size x 64      |
| Couche recurrente | 64 -> 128 (2 layers) |
| Dropout           | 0.3                  |
| Linear            | 128 -> 4 (classes)   |

### Differences entre les architectures

| Critere            | RNN           | LSTM       | GRU           |
|--------------------|---------------|------------|---------------|
| Complexite         | Faible        | Elevee     | Moyenne       |
| Parametres         | Le moins      | Le plus    | Intermediaire |
| Memoire long terme | Mauvaise      | Tres bonne | Bonne         |
| Vanishing gradient | Problematiche | Resolu     | Resolu        |
| Vitesse            | Rapide        | Lente      | Intermediaire |

## 3.5 Entrainement et Comparaison

Les trois modeles sont entraines pendant 15 epochs avec :

- Optimiseur : Adam (lr=0.001)
- Fonction de cout : CrossEntropyLoss
- Gradient clipping (max\_norm=1.0)
- Batch size : 64

Le gradient clipping est essentiel pour stabiliser l'entrainement des modeles recurrents et eviter l'explosion du gradient.

Les resultats montrent generalement que le LSTM et le GRU obtiennent de meilleures performances que le RNN simple, grace a leurs mecanismes de portes (gates) qui controlent le flux d'information.

L'evaluation detaillee inclut le classification report (precision, recall, f1 par classe), les matrices de confusion, et les courbes de loss/accuracy.

### Tableau Comparatif des Performances

| Modele | Parametres | Train Acc | Test Acc | Train Loss |
|--------|------------|-----------|----------|------------|
| RNN    | 1,122,180  | 43.70%    | 28.69%   | 1.0833     |
| LSTM   | 1,295,748  | 50.19%    | 42.23%   | 1.0858     |
| GRU    | 1,237,892  | 59.16%    | 47.75%   | 0.9763     |

## 3.6 Mini Modele Seq2Seq

Un modele Seq2Seq (Sequence-to-Sequence) est implemente avec un encodeur et un decodeur bases sur LSTM.

## Architecture

Encodeur : Embedding + LSTM (32 dim, 64 hidden)

Decodeur : Embedding + LSTM (32 dim, 64 hidden) + Linear

Teacher forcing ratio : 0.5

## Tache

La tache choisie est l'inversion de sequences de nombres (1-9). Par exemple :

- Entree : [3, 7, 1, 5, 2]
- Sortie : [2, 5, 1, 7, 3]

Le modele est entraine sur 5 000 sequences et teste sur 500. Apres 30 epochs, il atteint une accuracy par token elevee, montrant la capacite du Seq2Seq a apprendre des transformations sequentielles.

## 3.7 Analyse Critique et Conclusion

Avantages des LSTM / GRU :

- Bonne gestion des dependances a long terme
- Mecanisme de portes controlant le flux d'information
- Reduction significative du vanishing gradient
- Le GRU est plus leger que le LSTM avec des performances comparables

Limites :

- Temps d'entrainement eleve
- Traitement sequentiel (pas de parallelisation comme les Transformers)
- Difficultes sur les tres longues sequences
- Sensibilite aux hyperparametres

## Partie - Conclusion Generale

---

Ce projet de Deep Learning couvre trois familles fondamentales de reseaux de neurones profonds :

1. Les MLP (Perceptrons Multicouches) pour les donnees tabulaires, demontrant l'importance du preprocessing et de l'initialisation des poids.
2. Les CNN (Reseaux de Neurones Convolutionnels) pour la vision par ordinateur, montrant la puissance de l'extraction automatique de features spatiales.
3. Les RNN/LSTM/GRU pour les donnees sequentielles textuelles, illustrant les avantages des mecanismes de memoire pour la classification de texte, ainsi que le paradigme Seq2Seq pour les transformations sequence-a-sequence.

Chaque partie utilise des datasets reels et des methodologies rigoureuses d'evaluation, incluant des metriques variees (accuracy, precision, recall, F1), des visualisations (courbes de loss/accuracy, matrices de confusion), et des comparaisons entre differentes configurations.

Les outils utilises (PyTorch, scikit-learn, NumPy, Matplotlib) constituent un ecosysteme complet et professionnel pour le Deep Learning en Python.